

```

int main() {
    Hero adi;
    adi.level = 'A';
    adi.name = "Spiderman";
    adi.setHealth(100);

    cout << adi.level << endl;
    cout << adi.name << endl;
    cout << adi.getHealth << endl;
}

```

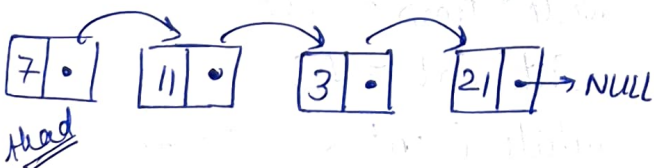
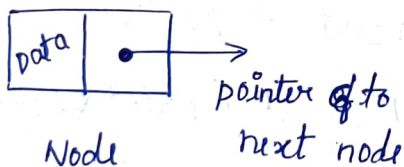
o/p : A
Spiderman
100

setHealth() → setter (void)
getHealth() → getter (returns a value)

lec 44

Linked list

→ Linear data structure (collection of Nodes)



NOTE

→ Vector is not optimal to inc ↑ size during run time.

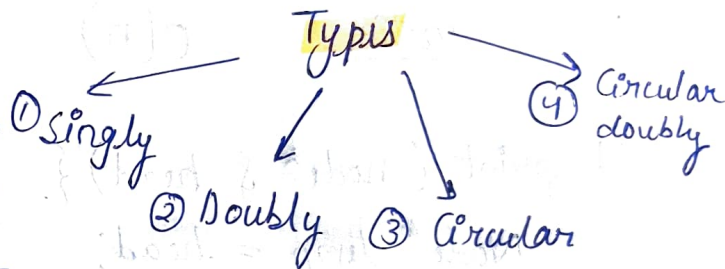
Advantages

- 1) Grow/Shrink optimally during runtime (Dynamic)
- 2) Insertion / Deletion → $O(1)$
- 3) No memory wastage.

Disadv

- 1) Uses more memory to store pointer.
- 2) Traversal → $O(n)$
- 3) Complex Implementation.
- 4) Not suited for small dataset.

Types



○

Implementation

```

Class Node {
public:
    int Data;
    Node* next;
};

```

```

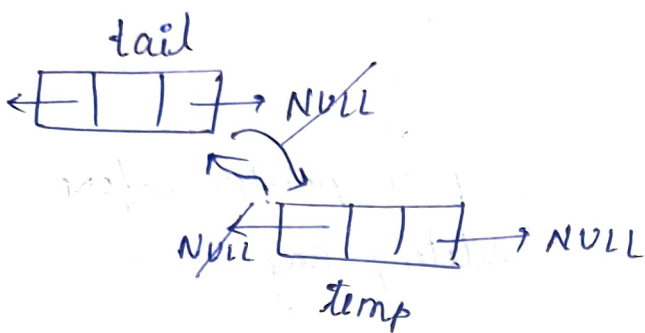
int main() {
    Node* node1 = new Node;
    node1->data = 100;
    node1->next = NULL;
}

```

return 0;

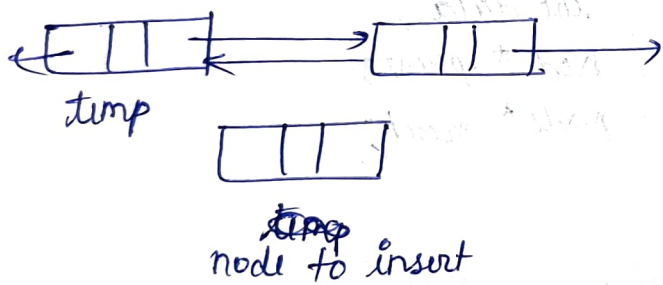
}

Insert at tail

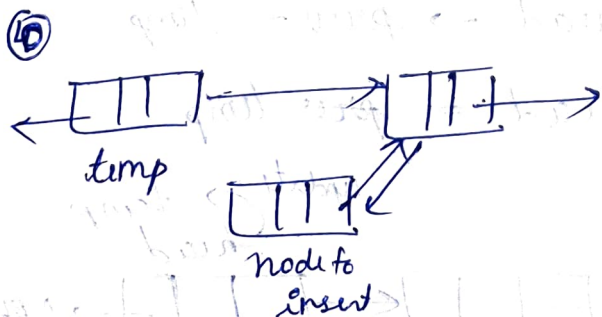


- ① Create temp Node
- ② $\text{tail} \rightarrow \text{next} = \text{temp}$
- ③ $\text{temp} \rightarrow \text{prev} = \text{tail}$
- ④ $\text{tail} = \text{temp}$:

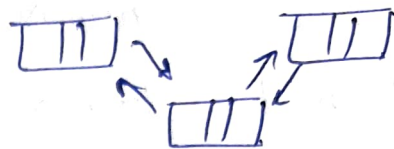
Insert at middle



- ① Create Node
- ② $\text{node to insert} \rightarrow \text{Next} = \text{temp} \rightarrow \text{next}$
- ③ $\text{temp} \rightarrow \text{next} \rightarrow \text{prev} = \text{node to insert}$



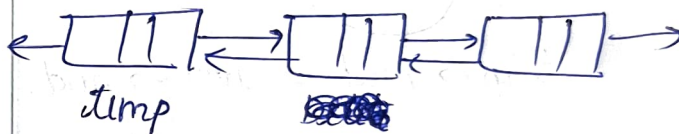
- ④ $\text{temp} \rightarrow \text{next} = \text{node to insert}$
- $\text{node to insert} \rightarrow \text{prev} = \text{temp}$:



If Node Head is NULL

- 1) Create new Node temp
- 2) $\text{head} = \text{temp}$

Deletion in DLL at head

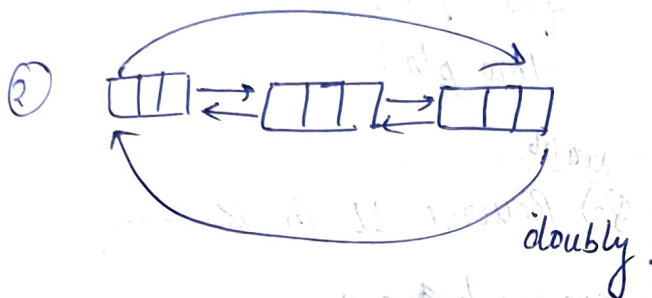
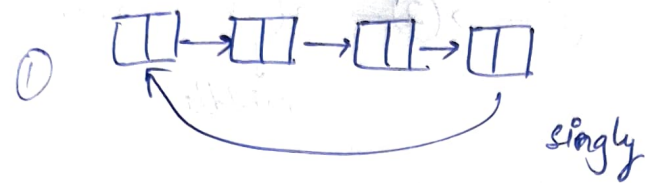


- ① $\text{temp} \rightarrow \text{next} \rightarrow \text{prev} = \text{NULL}$
- ② $\text{head} = \text{temp} \rightarrow \text{next}$
- ③ $\text{temp} \rightarrow \text{next} = \text{NULL}$
- ④ Delete temp

Deleting middle

- ① $\text{curr} \rightarrow \text{prev} = \text{NULL}$
- ② $\text{prev} \rightarrow \text{next} = \text{curr} \rightarrow \text{next}$
- ③ $\text{curr} \rightarrow \text{next} = \text{NULL}$
- ④ delete curr.

Circular Linked List



we don't use head, we use only ~~tail~~ tail pointer (we access head from $\text{tail} \rightarrow \text{next}$).

Traversal

```
void print (Node* &tail) {
```

```
    Node* temp = tail;
```

```
    while (tail->next != temp)
```

```
    do {
```

```
        cout << tail->data << " ";
```

```
        tail = tail->next;
```

```
    } while (tail != temp)
```

```
}
```

```
void InsertNode (Node* &tail,
```

```
int element, int d) {
```

```
    if (tail == NULL) {
```

```
        Node* newNode = new Node
```

```
(d);
```

```
        tail = newNode;
```

```
        newNode->next = newNode;
```

```
    } else {
```

```
        Node* curr = tail;
```

```
        while (curr->data !=
```

```
            element) {
```

```
            curr = curr->next;
```

```
        }
```

```
        Node* temp = new Node(d);
```

```
        temp->next = curr->next;
```

```
        curr->next = temp;
```

```
    }
```

```
}
```

uc 45

Q. Reverse a Linked list

soln 1) changing all pointers to prev nodes using two pointers prev and curr.



```
node* prev = NULL;
```

* store forward

logic

node* prev = NULL;

~~node~~
node* curr = head;

node* fwd = curr → next;

while (curr → next != NULL) {

fwd = curr → next;

curr → next = prev;

prev = curr;

curr = fwd;

}

using recursion

void reverse (Node* &head,
Node* prev, Node* curr) {

// base

if (curr == NULL) {

head = prev;

return;

}

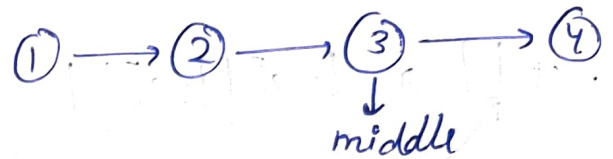
Node* forward = curr → next;

reverse (head, forward, curr)

curr → next = prev;

}

Q-) Middle of the linked list



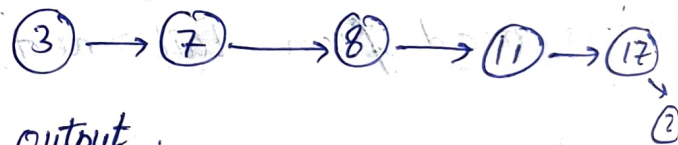
fast ptr;

slow ptr;

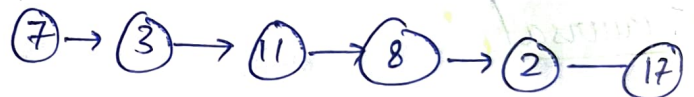
Q-) Reverse LL in k group.

approach : Recursion

input k=2



output



Node* kReverse (Node* head, int k)

// base call

if (head == NULL) {

return NULL;

}

// step 1: reverse first k nodes

Node* next = NULL;

Node* curr = head;

Node* prev = NULL;

int count = 0;

while (curr != NULL && count < k) {

next = curr → next;

curr → next = prev;

prev = curr;

curr = next;

count++;

}

Q.) Circular LL or not.

```
bool isCircular (Node* head) {  
    if (head == NULL) {  
        return NULL true;  
    }  
    Node* temp = head → next;  
    while (temp != NULL &&  
           temp != head) {  
        temp = temp → next;  
    }  
    if (temp == head) {  
        return head true;  
    }  
    return false;  
}
```

This only detects Circular LL not loops.

approach two - using map.

map < Node*, bool visited >

step 2: Recursion

```
if (next != NULL) {  
    head → next = kReverse(next, k);  
}
```

step 3: return head of ~~prev~~ reversed list

return prev;

}

Q.) Detect and Remove loop. (lc 47)

.) Detecting a loop using map

```
bool detectLoop (Node* head) {  
    if (head == NULL)  
        return false;  
    map < Node*, bool > visited;  
    Node* temp = head;  
    while (temp != NULL) {  
        if (visited[temp] == true) {  
            return true;  
        }  
        temp = temp → next;  
        visited[temp] = true;  
        temp = temp → next;  
    }  
    return false;  
}
```

.) Floyd cycle detection in LL.
(fast and slow pointers).

```
bool FloydDetectLoop (Node* head) {  
    if (head == NULL) {  
        return false;  
    }  
    Node* slow = head;  
    Node* fast = head;  
    while (slow != NULL && fast != NULL) {  
        fast = fast → next;  
        if (fast != NULL) {  
            fast = fast → next;  
        }  
        slow = slow → next;  
    }
```



```

if (slow == fast) {
    return true;
}
}

```

```

return false;
}

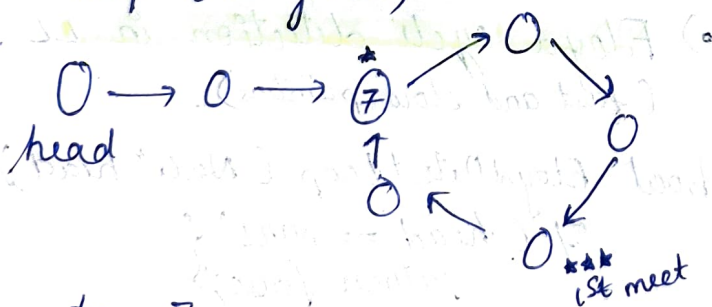
```

time complexity = $O(n)$

space comp = $O(1)$.

it sure that fast & slow pointers will meet because after each iteration distance between them reduces by 1 (if both are present in cycle).

Q. Return starting Node of the loop (using FLD)



return 7

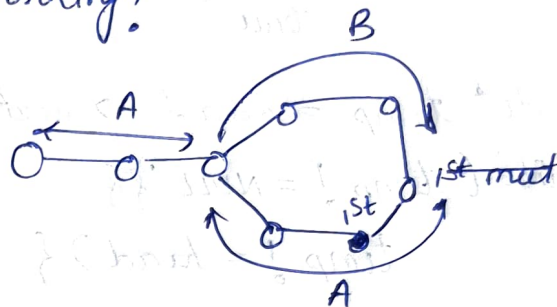
approach

1) Using Floyd loop detection, stop when slow and fast meet for the 1st time

- reduce speed of fast (1)
- ~~slow~~ slow $\rightarrow$ head

2) Return the data of Node when slow and fast meets for 2nd time

Working?



to complete cycle B, fast requires 'A' distance to cover.

Proof

$$\text{dist}(\text{fast}) = 2 \text{dist}(\text{slow})$$

$$A + xC + B = 2(A + yC + B)$$

$x \rightarrow$ cycles completed by fast
 $y \rightarrow$ " " " slow

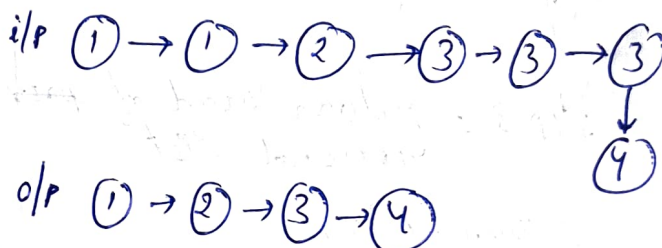
$$A + B = (x - 2y)C$$

$\downarrow$
 $K C$
'K' cycles

lec 48

Q. Remove duplicates from sorted list

approach:



logic
→ traversal through LL &
delete if ($curr \rightarrow val == curr$
 $\downarrow$
 $next$
 $\downarrow$
 val).

```
Node* UniqueSortList (head) {
```

3

```
while (curr != NULL) {
```

```
Node* next_next = curr->next->next;
```

delete (note Todelete);

```
curr → next = next → next;
} else {
```

curr = curr → next

3

return head;

3

④ → ② → ⑤ → ② → ④ → ③
↓
②

(4) → (2) → (5) → (3)

using brute force (two pointers)

$$O \rightarrow O \rightarrow O \rightarrow O \rightarrow O$$

$$\text{curr} \quad \text{temp} \rightarrow$$

subili (corr != NULL) {

~~if (curr == temp)~~

```
if (curr->data == temp->data){
```

delete (temp) :

3

T.C = $O(n^2)$

approach 2

using map $T.C O(n)$

S.C (no. of unique elements)

S.C ($O(n)$)

approach 3: $O(n \log n)$

1) Sort LL

2) Remove duplicate $O(n)$

lc 49

Q. Sort LL of 0s, 1s and 2s.

i/p $1 \rightarrow 2 \rightarrow 0 \rightarrow 0 \rightarrow 1$

o/p $0 \rightarrow 0 \rightarrow 1 \rightarrow 1 \rightarrow 2$

Approach: count 0, 1 and 2 and make a new LL (traversal)

approach 2: Changing links

create 3 LL of 0, 1 and 2

zero - $\text{dummy} \rightarrow 0 \rightarrow 0$

one - $\text{dummy} \rightarrow 1 \rightarrow 1$

two - $\text{dummy} \rightarrow 2$
dummy Node.

and merge $0 \rightarrow 1 \rightarrow 2$

$T(c) = O(n)$ $SC = O(1)$

Node* SortList (Node* Head) {

Node* zeroHead = new Node(-1);

Node* zeroTail = zeroHead;

Node* oneHead = new Node(-1);

Node* oneTail = oneHead;

Node* twoHead = new Node(-1);

Node* twoTail = twoHead;

Node* curr = Head;

while (curr != NULL) {

int val = curr->data;

if (value == 0) {

insertAtTail (zeroTail, curr);

} else if (value == 1) {

insertAtTail (oneTail, curr);

} else {

insertAtTail (twoTail, curr);

}

curr = curr->next;

}

// Created LL, now merging them.

if (oneHead->next != NULL) {

zeroTail->next = oneHead->next;

} else {

zeroTail->next = twoHead->next;

}

oneTail->next = twoHead->next;

twoTail->next = NULL;

// setup head

head = zeroHead->next;

head = zeroHead->next;

delete zeroHead;

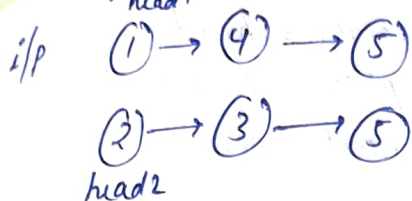
delete oneHead;

delete twoHead;

return head;

}

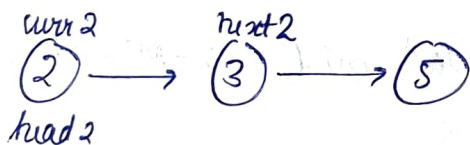
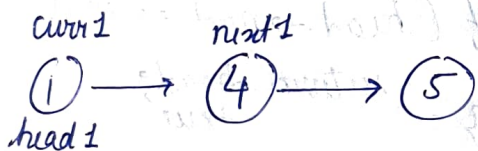
Q. Merge 2 Sorted Linked List.



approach Nodes of List 2 into list 1 by comparing Node → data.

code.

Assume first list whose 1st Node has least value.



void solve (Node <int>* first, Node <int>* second) {

Node* curr1 = first;

Node* next1 = curr1 → next;

Node* curr2 = second;

Node* next2 = curr2 → next;

while (next1 != NULL && curr2 != NULL) {

if (curr2 → data < curr1 → data) {

{ curr1 = next1; next1 = next1 → next; }

curr1 → next = curr2;
curr2 → next = next;
curr1 = curr2
curr2 = next2;

} else {

curr1 = next1;
next1 = next1 → next;

if (next1 == NULL) {

curr1 → next = curr2;
return first;

}

}

return first;

}

Node <int>* sortTwoLists (

Node <int>* first, Node <int>* second)

{ if (first == NULL)

return second;

if (second == NULL) {

return first;

if (first → data <= second → data) {

solve (first, second);

} else {

solve (second, first);

}

}

Q. Check palindrome in LL.

l/p $1 \rightarrow 2 \rightarrow 2 \rightarrow 1$

o/p True.

approach 1:

→ Convert LL into arr[] or string and check.

T.C = $O(n)$

S.C = $O(n)$

approach 2:

- ① find middle of LL;
- ② Reverse LL after middle
- ③ Compare both halves
- ④ Repeat step 2.

Node* getMid (Node* head) {

Node* slow = head;

Node* fast = head → next;

while (fast != NULL &&
fast → next != NULL) {

fast = fast → next → next;
slow = slow → next;

}

return slow;

}

Node* reverse (Node* head) {

Node* curr = head;

Node* prev = NULL;

Node next = NULL;

while (curr != NULL) {

next = curr → next;

curr → next → prev;

prev = curr;

curr = next;

}

return prev;

}

bool Palindrome (Node* head) {

if (head → next == NULL) {

return true;

}

1) Node* mid = getMid (head);

2) Node* temp = mid → next;
mid → next = reverse (temp);

3) Node* head1 = head;
Node* head2 = mid temp;

4) while (head2 != NULL) {
if (head1 → data != head2 → data) {

return false;

}

head1 = head1 → next;

head2 = head2 → next;

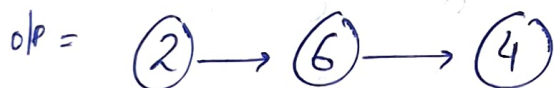
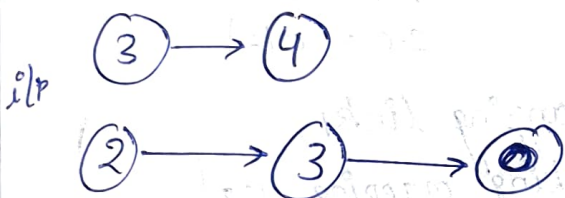
}


```

4) temp = middle → next;
   middle → next = reverse(temp);
   return true;
}

```

Q. Add 2 Numbers



approach

- ① Reverse LL
- ② Add from left with carry
- ③ Reverse ans and print.

code for only step ②

```

struct Node* add (Node* first,
Node* second) {
    int carry = 0;
    Node* ansHead = NULL;
    Node* ansHead = NULL;
    ansTail

```

```

while (first != NULL || second != NULL
|| carry != 0) {

```

```

    int val1 = 0;
    if (first != NULL) val1 = first->data;

```

```

    int val2 = 0;
    if (second != NULL) val2 = second->data;

```

```

    int sum = carry + val1 + val2;

```

```

    int digit = sum / 10;

```

```

    int carry = sum / 10;

```

```

    insertAtTail (ansHead, ansTail,
digit);

```

```

    if (first != NULL) {
        first = first → next;
    }

```

```

    if (second != NULL) {
        second = second → next;
    }

```

```

    return ansHead;
}

```

T.C = $O(M+N)$

S.L = $O(\max(M, N))$

lc 52

Q.) Clone a LL with Random Pointers.
(next and Random pointer).

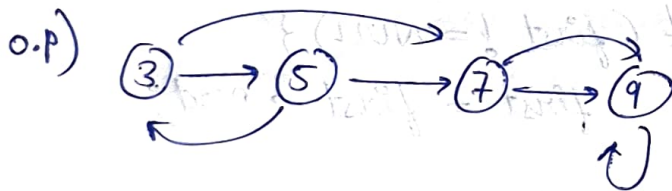
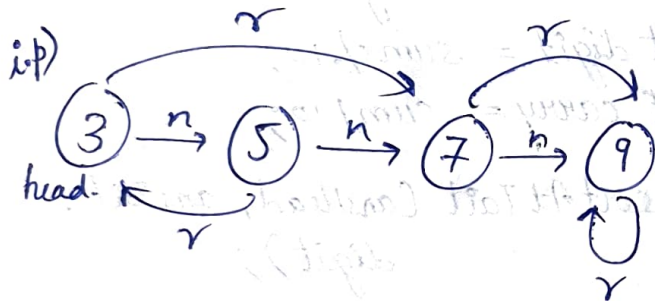
struct Node {

int data;

node* next;

node* random;

}

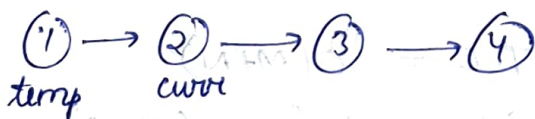


Just clone

approach 1

T.C = $O(n^2)$

- 1) create a new LL with next
- 2) for random pointer check with two pointer approach



if (curr -> random == temp)

else

temp = temp -> next;

approach 2

T.C = $O(n)$

S.C = $O(n)$ map

using map

mapping original LL & cloned LL.

then

(clone -> random = original -> random)

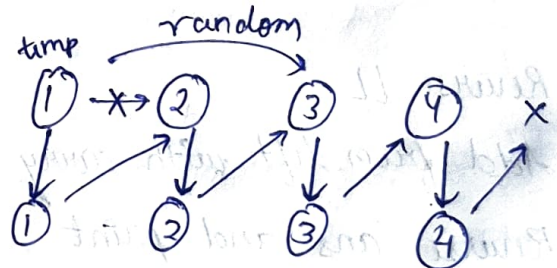
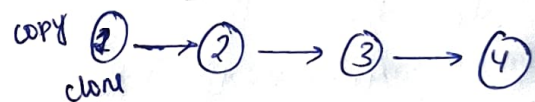
approach 3

T.C = $O(n)$

S.C = $O(1)$

changing links.

making mapping real



step 1

- 1) create a clone list
- 2) Add clone nodes between original nodes.
- 3) Random pointer

(temp -> next -> random) = temp -> random -> next;

4) Revert changes of step 2.

code

```
void InsertAtTail (Node* &head,  
Node* &tail, int d) {  
    Node* newNode = new Node(d);  
    if (head == NULL) {  
        head = newNode;  
        tail = newNode;  
    } else {  
        tail->next = newNode;  
        tail = newNode;  
    }  
}
```

```
Node* CopyList (Node* head) {  
    Node* originalNode = head;  
    Node* cloneNode = NULL;  
  
    Node* temp = head;  
    while (temp != NULL) {  
        InsertAtTail (&head, &tail,  
            temp->data);  
        temp = temp->next;  
    }  
}
```

②

```
Node* CopyList (Node* head) {
```

```
① Node* cloneHead = NULL;
```

```
Node* cloneTail = NULL;
```

```
Node* temp = head;
```

```
while (temp != NULL) {  
    InsertAtTail (&cloneHead,  
        &cloneTail, temp->data);  
    temp = temp->next;  
}
```

②

```
Node* originalNode = head;  
Node* cloneNode = cloneHead;
```

```
while (originalNode != NULL &&  
    originalNode != NULL) {
```

```
Node* next = originalNode->  
    next;
```

```
originalNode->next = cloneNode;  
originalNode = next;
```

```
next = cloneNode->next;  
cloneNode->next = originalNode;  
cloneNode = next;
```

```
}
```

③

```
Node* temp = head;
```

```
while (temp != NULL) {
```

```
if (temp->next != NULL) {
```

```
(temp->next->random) =
```

```
temp->random;
```

```
temp->random->next;
```

```
temp->random;
```

```
}
```

```

temp = next →
temp → next → next;
}

```

④

```

original Node = head;
clone Node = clone Head;
while (original Node != NULL &&
       clone Node != NULL) {
    original Node → next = clone →
                                next;
    original Node = original → next;
    if (original != NULL) {
        clone → next = original → next;
        clone = clone → next;
    }
}

```

lec 53

Merge Sort in LL

if ① → ⑤ → ③ → ④ → ②

① → ② → ③ → ④ → ⑤

approach

changing links.

same as array

↳ divide

↳ sort

↳ merge to sorted LL.

```

Node * find mid (Node * head) {

```

```

    Node * fast = head;

```

```

    Node * slow = head;

```

```

    while (fast != NULL) {
        fast = fast → next;
        if (fast != NULL) {
            fast = fast → next;
        }
    }

```

```

    slow = slow → next;

```

```

}
return slow;
}

```

```

Node * merge (Node * left, Node * right)

```

```

    if (left == NULL) {
        return right;
    }

```

```

    if (right == NULL) {
        return left;
    }

```

// dummy node

```

Node * ans = new Node (-1);

```

```

Node * temp = ans;

```

```

while (left != NULL && right != NULL)

```

```

    if (left → data < right → data)

```

```

        temp → next = left;

```

```

        temp = left;

```

```

        left = left → next;

```

```

    } else {

```

```

        temp → next = right;

```

```

        temp = right;

```

```

    } right = right → next;
}

```



```
while (right != NULL) {  
    temp → next = right;  
    temp = right;  
    right = right → next;  
}
```

```
while (left != NULL) {  
    = left  
    = left;  
    left = left → next;  
}
```

```
return ans → next;
```

```
}
```

```
Node* mergeSort (Node* head) {
```

```
    if (head == NULL || head →  
        next == NULL) {  
        return head;  
    }
```

```
    Node* mid = getmid (head);
```

```
    Node* left = head;
```

```
    Node* right = mid + 1;
```

```
    mid → next = NULL;
```

```
    left = mergeSort (left);
```

```
    right = mergeSort (right);
```

```
    Node* result = merge (left, right);
```

```
    return result;
```

```
}
```